

Instituto Tecnológico Superior de Río verde SLP.

TEMA:

Semana 11

ALUMNO:

Brayan Ivanov Martínez Martínez

MATERIA:

programacion

DOCENTE:

Jesus collazo

GRUPO

2ª A

Fecha

4/05/2026

Introducción (¿qué es herencia?)

La herencia es cuando una clase toma cosas de otra clase

Es como en la vida real, donde los hijos heredan cosas de sus papás

Sirve para no repetir código y hacer el programa más fácil

Por ejemplo, si todos los personajes tienen nombre y vida, eso se pone en una sola clase y las demás lo usan

Main.cpp

Aquí se crean los objetos

y se prueba el programa

Se ejecutan las funciones

para ver cómo funciona cada personaje.

```
semana11 > G main.cpp
1  #include <iostream>
2  #include "personajes.h"
3  using namespace std;
4
5  int main() {
6      cout << "\n SISTEMA DE PERSONAJES RPG \n";
7
8      Guerrero g("maximo", 10, 300, 50);
9      cout << ">> Info del Guerrero:\n";
10     g.mostrarInfo();
11     g.atacar();
12     cout << "\n";
13
14     Mago m("oz", 8, 180, "rayo");
15     cout << ">> Info del Mago:\n";
16     m.mostrarInfo();
17     m.atacar();
18     cout << "\n";
19
20     Arquero a("barton", 7, 220, 80);
21     cout << ">> Info del Arquero:\n";
22     a.mostrarInfo();
23     a.atacar();
24     cout << "\n";
25
26     PersonajeEspecial pe("elsa", 15, 500, 70, "hielo");
27     cout << ">> Info del Personaje Especial:\n";
28     pe.mostrarInfoEspecial();
29     pe.atacar();
30
31     return 0;
32 }
33
```

```

semana11 > g++ personajes.cpp
1  #include "personajes.h"
2  #include <iostream>
3  using namespace std;
4
5
6  Personaje::Personaje(string nombre, int nivel, int vida) {
7      this->nombre = nombre;
8      this->nivel = nivel;
9      this->vida = vida;
10 }
11
12 void Personaje::mostrarInfo() {
13
14     cout << " Nombre : " << nombre << "\n";
15     cout << " Nivel : " << nivel << "\n";
16     cout << " Vida : " << vida << "\n";
17 }
18
19
20 void Personaje::atacar() {
21     cout << nombre << " lanza un ataque basico!\n";
22 }
23
24 string Personaje::getNombre() {
25     return nombre;
26 }
27
28 Guerrero::Guerrero(string nombre, int nivel, int vida, int fuerzaExtra)
29     : Personaje(nombre, nivel, vida) {
30     this->fuerzaExtra = fuerzaExtra;
31 }
32
33 void Guerrero::atacar() {
34     cout << nombre << " golpea con su espada! (+\"
35     | << fuerzaExtra << " de dano extra) CRASH!\n";
36 }

```

```

semana11 > g++ personajes.cpp
39 Mago::Mago(string nombre, int nivel, int vida, string elemento)
40     : Personaje(nombre, nivel, vida) {
41     this->elemento = elemento;
42 }
43
44 void Mago::atacar() {
45     cout << nombre << " lanza un hechizo de " << elemento
46     | << "! BOOM! Critico! El enemigo esta congelado!\n";
47 }
48
49 Arquero::Arquero(string nombre, int nivel, int vida, int alcance)
50     : Personaje(nombre, nivel, vida) {
51     this->alcance = alcance;
52 }
53
54 void Arquero::atacar() {
55     cout << nombre << " dispara una flecha desde "
56     | << alcance << " Critico! Impacto directo!\n";
57 }
58
59 PersonajeEspecial::PersonajeEspecial(string nombre, int nivel, int vida,
60     | | | | | | | | int fuerzaExtra, string elemento)
61     : Guerrero(nombre, nivel, vida, fuerzaExtra),
62     | Mago(nombre, nivel, vida, elemento) {}
63
64 void PersonajeEspecial::atacar() {
65     Guerrero::atacar();
66     Mago::atacar();
67     cout << ">>> COMBO BRUTAL activado! El enemigo no sabe que paso!\n";
68 }
69
70 void PersonajeEspecial::mostrarInfoEspecial() {
71     cout << " [PERSONAJE ESPECIAL]\n";
72     Guerrero::mostrarInfo();
73     cout << " Tipo : Guerrero-Mago\n";
74 }

```

Personajes.cpp

Aquí se implementan las funciones de las clases como mostrar datos o acciones especiales de cada personaje.

```

semana11 > C personajes.h
1  #ifndef PERSONAJES_H
2  #define PERSONAJES_H
3
4  #include <string>
5  using namespace std;
6
7  class Personaje {
8  protected:
9      string nombre;
10     int nivel;
11     int vida;
12
13 public:
14     Personaje(string nombre, int nivel, int vida);
15     void mostrarInfo();
16     virtual void atacar();
17     string getNombre();
18 };
19
20 class Guerrero : public Personaje {
21 private:
22     int fuerzaExtra;
23
24 public:
25     Guerrero(string nombre, int nivel, int vida, int fuerzaExtra);
26     void atacar() override;
27 };
28
29 class Mago : public Personaje {
30 private:
31     string elemento;
32
33 public:
34     Mago(string nombre, int nivel, int vida, string elemento);
35     void atacar() override;
36 };
37
38 class Arquero : public Personaje {
39 private:
40     int alcance;
41
42 public:
43     Arquero(string nombre, int nivel, int vida, int alcance);
44     void atacar() override;
45 };
46
47
48 class PersonajeEspecial : public Guerrero, public Mago {
49 public:
50     PersonajeEspecial(string nombre, int nivel, int vida,
51                       |   |   |   |   int fuerzaExtra, string elemento);
52     void atacar() override;
53     void mostrarInfoEspecial();
54 };
55
56 #endif
57

```

Personajes.h

En este archivo se definen las clases. Aquí se crea la clase base (Personaje) con atributos generales como nombre y vida. También se crean las clases derivadas que heredan de la clase base.

```
ivano@ivanov MINGW64 ~/OneDrive/Escritorio/p1/semana11
$ g++ main.cpp personajes.cpp -o programa

ivano@ivanov MINGW64 ~/OneDrive/Escritorio/p1/semana11
$ |

ivano@ivanov MINGW64 ~/OneDrive/Escritorio/p1/semana11
$ ./programa

SISTEMA DE PERSONAJES RPG
>> Info del Guerrero:
  Nombre : maximo
  Nivel  : 10
  Vida   : 300
maximo golpea con su espada! (+50 de dano extra)

>> Info del Mago:
  Nombre : oz
  Nivel  : 9
  Vida   : 150
oz lanza un hechizo rayo Critico! El enemigo esta congelado!

>> Info del Arquero:
  Nombre : barton
  Nivel  : 7
  Vida   : 250
barton dispara una flecha dano 80 Critico! Impacto directo!

Nivel : 9
Vida : 150
z lanza un hechizo rayo Critico! El enemigo esta congelado!

> Info del Arquero:
  Nombre : barton
  Nivel  : 7
  Vida   : 250
arton dispara una flecha dano 80 Critico! Impacto directo!

> Info del Personaje Especial:
[PERSONAJE ESPECIAL]
  Nombre : elsa
  Nivel  : 15
  Vida   : 500
  Tipo   : Guerrero-Mago
elsa golpea con su espada! (+70 de dano extra)
elsa lanza un hechizo hielo Critico! El enemigo esta congelado!
>> COMBO BRUTAL activado! El enemigo no sabe que paso!

ivano@ivanov MINGW64 ~/OneDrive/Escritorio/p1/semana11
|
```

Esta imagen muestra la ejecución del programa donde se crean los personajes y se muestran sus acciones.

¿Cómo usaron la herencia?

Se usó herencia creando una clase base llamada Personaje que tiene datos generales como nombre y vida

Después se crearon clases derivadas como guerrero, mago, etc. que heredan esos datos pero también tienen cosas propias, como ataques o habilidades Así se evita repetir código y todo está mejor organizado

Diferencia entre clase base y derivada

Clase base:

Es la clase principal Tiene las características generales que todos comparten

Clase derivada:

Es una clase que hereda de la base Tiene lo mismo que la base, pero además agrega o cambia cosas.

Personaje (base) → tiene nombre y vida

Guerrero (derivada) → tiene nombre, vida y ataque con espada

6. Conclusiones

La herencia es muy útil porque permite reutilizar código y organizar mejor los programas con esto no tenemos que escribir lo mismo varias veces también hace que el programa sea más fácil de entender y modificar

¿Cuál sería la clase principal?

La clase principal sería Personaje, porque de ahí salen todos los demás.

¿Qué clases derivadas existirían?

Guerrero, Mago, Arquero

Porque todos son personajes pero hacen cosas diferentes.

Definir atributos

¿Qué datos tendría la clase base?

La clase base tendría cosas básicas como: Nombre, vida

Porque todos los personajes tienen eso

¿Qué datos extra tendría cada tipo?

Guerrero: fuerza o ataque

Mago: magia o poder mágico

Arquero: precisión o velocidad

Definir comportamiento

¿Qué métodos tendría la clase base?

mostrar datos, atacar

¿Qué métodos cambiarían en cada clase?

El ataque cambia en cada uno:

Guerrero = ataque fuerte

Mago = ataque con magia

Arquero = ataque a distancia

Explicar la herencia

¿Quién hereda de quién?

Las clases como Guerrero, Mago y Arquero heredan de Personaje.

¿Por qué decidiste esa estructura?

Porque todos tienen cosas en común es mejor ponerlo en una sola clase y no repetir código

Diseñar el personaje especial

¿Cómo combinarías habilidades?

Podría ser un personaje que tenga fuerza y magia al mismo tiempo, o sea que pueda atacar de las dos formas.

¿Usarías herencia múltiple o no? ¿por qué?

No usaría herencia múltiple porque puede complicar el código, Mejor hago una sola clase que combine las habilidades para que sea más fácil.

